

Rendir.ai

De una foto de tus apuntes a un simulacro al estilo de TU profesor.

Jorge Luis Valer Osis · Solo founder

Proyecto Final — Data Science con Python 2026-I · Universidad del Pacífico

Pitch · 14 secciones estilo Y Combinator

2 • Founder

- **Jorge Luis Valer Osis** — solo founder. Estudiante UP: **soy el usuario, vivo el problema** (*founder-market fit*).
- Construido con **agentes de IA: Claude Code** (CTO backend), Claude/Codex (frontend), crewAI (pipeline).
- Una persona + agentes de IA = una startup funcional en semanas.

3 · Problema

- **Quién:** universitarios en cursos con **examen de desarrollo**.
- Estudian **“a ciegas”**: no saben qué **estilo** de preguntas tomará el profe.
- Armar simulacros a mano toma **horas**; ChatGPT genérico falla (apuntes a mano/pizarra + preguntas genéricas).
- *“No sabes exactamente qué te va a venir...”* — **validado con 7 entrevistas**.

4 · Solución & Insight

- Subes una **foto de apuntes** (+ opcional un **examen pasado**) → **simulacro al estilo de ESE profesor.**
- **Insight:** en LatAm los exámenes son de desarrollo y **cada profesor tiene un estilo repetitivo y heredable.**
- Modelamos al **docente**, no a “un curso genérico”.

5 · Why now

- LLMs baratos + **Claude visión** lee manuscrito/pizarra (donde el OCR clásico falla).
- **PaddleOCR** gratis para material impreso (sin costo de tokens).
- **Claude Code** deja a una persona construir el producto en horas.
- La ventana tecnológica es **ahora**.

6 • Mercado (TAM / SAM / SOM)

TAM

30.9M

universitarios en LatAm
(CLACSO)

SAM

720K

Perú, carreras con
examen de desarrollo
(SUNEDU)

SOM

5,000

año 1 (UP + Lima)
+ *pilotos B2B*

7 · Competencia & Moat

- vs **ChatGPT/NotebookLM**: genéricos, leen mal la foto, no modelan al profe. vs **hacerlo a mano**: horas.
- **Moat**: dataset propio por (*profesor* $\times$ *curso*). Cada examen subido **mejora la predicción** $\rightarrow$ efecto de red.
- **Validado**: 4/7 entrevistados heredan exámenes pasados de terceros.

8 · Producto — demo + arquitectura

- **Demo desplegado** (URL pública, sin instalar nada).
- **Lector híbrido:** Claude visión (manuscrito) + PaddleOCR (impreso) → simulacro → PDF.
- **Arquitectura:** Streamlit → FastAPI/Render → Claude API / PaddleOCR.
- **2 herramientas del curso:** Claude API + PaddleOCR.

1 Modelo de Diamond & Dybvig: provisión de liquidez y bank runs

[5 puntos] (Bank runs! - Diamond & Dybvig, 1983). Considere un banco que capta 100 depositantes que aportan 1 sol cada uno. La probabilidad de ser "impaciente" (consumir en $t=1$) es $p=25\%$, mientras que el resto son "pacientes" (consumen en $t=2$). El banco invierte en un proyecto con pagos $r_1=1$ y $r_2=2$. Responda: (a) Explique cómo el modelo de Diamond & Dybvig genera tanto un equilibrio eficiente como uno ineficiente, precisando el rol de las expectativas en la aparición de un bank run (1 punto). (b) Si el banco ofrece pagar 1.28 a los impacientes en $t=1$, ¿cuánto recibirán los pacientes en $t=2$? Muestre cuántas unidades del proyecto debe liquidar y los cálculos respectivos (2 puntos). (c) El mercado a las 8:00 am cree que el 30% de los depositantes reclamará en $t=1$. ¿Cuánto recibirá cada uno de los pacientes restantes en $t=2$ bajo este escenario? Muestre sus cálculos e interprete (2 puntos).

🔗 Evalúa: Comprensión del rol de las expectativas como profecías autocumplidas, capacidad de calcular pagos intertemporales mediante liquidación parcial del proyecto y manejo del trade-off liquidez-rentabilidad.

▼ Ver esquema de respuesta

(a) El modelo tiene dos equilibrios: uno eficiente (solo los impacientes retiran en $t=1$, los pacientes esperan a $t=2$ y obtienen el mayor pago) y uno ineficiente (bank run), donde si los pacientes creen que otros retirarán anticipadamente tienen incentivo a retirar también, forzando la liquidación de activos productivos a "fire sale"; las expectativas son autocumplidas. (b) 25 impacientes $\times 1.28 = 32$ soles; el banco liquida 32 unidades del proyecto (a $r_1=1$). Quedan 68 unidades invertidas que en $t=2$ rinden $68 \times 2 = 136$. Repartido entre 75 pacientes: $136/75 = 1.813$. (c) Si 30 retiran en $t=1$: $30 \times 1.28 = 38.4$; liquida 38.4 unidades; quedan $61.6 \times 2 = 123.2$; repartido entre 70 pacientes: $123.2/70 = 1.79$. El pago cae respecto a 1.813, lo que ilustra cómo mayores retiros anticipados deterioran el pago de los pacientes y pueden gatillar el pánico.

Simulacro generado al estilo del profesor

9 · Modelo de negocio & pricing

- **Freemium + B2B academias.** Free / **Plus US\$ 3.49/mes** (o US\$ 26/año) / **B2B desde US\$ 2 por alumno-ciclo.**
- **Contribution margin 58–86 %** (Haiku/Sonnet); impreso = US\$ o (PaddleOCR).
- **Se autofinancia con ~4–9 usuarios de pago** (punto de equilibrio año 1).

10 · Go-to-market

- **10:** mi salón / facultad en la UP.
- **100:** boca a boca en grupos de WhatsApp por curso + facultades UP.
- **1,000:** otras universidades de Lima + pilotos B2B con academias.

11 • Tracción / señales tempranas

- Demo **desplegado y funcionando** (URL pública).
- 7 entrevistas validan problema y moat.
- Un entrevistado **ya hace Rendir.ai a mano** con ChatGPT → demanda real.
- *En curso*: 3 usuarios prueban el demo.

12 · Roadmap

- **3 meses:** pulir flujo + sembrar dataset (*profesor* × *curso*) en 1 facultad UP.
- **6 meses:** piloto B2B con 1–2 academias; más universidades de Lima.
- **12 meses:** cobertura Perú + app móvil.

13 · Riesgos & mitigación

- **Técnico** (calidad del simulacro): feedback loop + fine-tune con exámenes reales.
- **Mercado** (disposición a pagar): freemium para adquirir + B2B donde sí hay presupuesto.
- **Ejecución** (solo founder): agentes de IA + foco en UN flujo que funcione.

14 · The ask

- Buscamos **100 estudiantes beta este ciclo + 1 academia piloto** para sembrar el dataset.
- Eso desbloquea el **moat** (*dataset por profesor $\times$ curso*) y las primeras señales de pago.

“Make something people want.” — Y Combinator